

Customizando o pipeline do spaCy

Trabalhando com Texto em Python I — Unidade 3

Modificar o pipeline, processar em lote, atributos personalizados, salvar em disco e mesclar sintagmas e entidades.

Adaptado de *Applied Language Technology* (Universidade de Helsinque) — applied-language-technology.mooc.fi

Objetivos de aprendizagem

Ao final desta unidade, você deverá saber:

- examinar e modificar o pipeline do spaCy;
- processar textos com eficiência;
- adicionar atributos personalizados a objetos do spaCy;
- salvar em disco os textos processados;
- mesclar sintagmas nominais e entidades nomeadas em tokens únicos.

Vamos começar importando a biblioteca spaCy e o módulo `displacy`, usado para desenhar árvores de dependência, e carregando um modelo de linguagem para o inglês:

Python

```
# Importa a biblioteca spaCy e o modulo displacy
from spacy import displacy
import spacy

# Carrega um modelo pequeno de lingua inglesa e o atribui a variavel 'nlp'
nlp = spacy.load('en_core_web_sm')

# Chama a variavel para examinar o objeto
nlp
```

Saída

```
<spacy.lang.en.English at 0x294522f10>
```

1. Modificando pipelines do spaCy

O objeto `Language` é, essencialmente, um **pipeline** que aplica um modelo de linguagem ao texto, executando as tarefas para as quais o modelo foi treinado. As tarefas realizadas dependem dos **componentes** presentes no pipeline.

Podemos examinar os componentes com o atributo `pipeline` de um objeto `Language`:

Python

```
nlp.pipeline
```

Saída

```
[('tok2vec', <spacy.pipeline.tok2vec.Tok2Vec at 0x294cf3dc0>),  
( 'tagger', <spacy.pipeline.tagger.Tagger at 0x294cf3ca0>),  
( 'parser', <spacy.pipeline.dep_parser.DependencyParser at 0x294b566d0>),  
( 'attribute_ruler', <spacy.pipeline.attributeruler.AttributeRuler at 0x295187680>),  
( 'lemmatizer', <spacy.lang.en.lemmatizer.EnglishLemmatizer at 0x295191540>),  
( 'ner', <spacy.pipeline.ner.EntityRecognizer at 0x294b56740>)]
```

Isso retorna um objeto `SimpleFrozenList` do spaCy, composto por **tuplas** com dois itens: o **nome** do componente (ex.: `tagger`) e o **componente** que executa a tarefa (ex.: `spacy.pipeline.tok2vec.Tok2Vec`).

Componentes como `tagger`, `parser`, `ner` e `lemmatizer` já são familiares da unidade anterior. Há dois novos:

- `tok2vec` mapeia os tokens para suas representações numéricas (veremos na Parte III).
- `attribute_ruler` aplica regras definidas pelo usuário aos tokens e adiciona essa informação como atributo, se solicitado.

Nota

A lista em `nlp.pipeline` **não** inclui o `Tokenizer`, porque todo texto precisa ser tokenizado para qualquer processamento ocorrer. Por isso ele fica no atributo `tokenizer`, e não em `pipeline`.

1.1 Excluindo componentes para ganhar desempenho

Todo componente tem um custo computacional. Se você não precisa da saída de um componente, não deve incluí-lo, pois o processamento fica mais lento. Para excluir, forneça o argumento `exclude` com os nomes dos componentes ao chamar `load()`:

Python

```
# Carrega um modelo pequeno de lingua inglesa, mas exclui o reconhecimento de  
# entidades nomeadas ('ner') e a analise de dependencias sintaticas ('parser').  
nlp = spacy.load('en_core_web_sm', exclude=['ner', 'parser'])  
  
# Examina os componentes ativos no objeto Language 'nlp'  
nlp.pipeline
```

Saída

```
[('tok2vec', <spacy.pipeline.tok2vec.Tok2Vec at 0x29514d760>),  
( 'tagger', <spacy.pipeline.tagger.Tagger at 0x296c7ba00>),  
( 'attribute_ruler', <spacy.pipeline.attributeruler.AttributeRuler at 0x296c6bb40>),  
( 'lemmatizer', <spacy.lang.en.lemmatizer.EnglishLemmatizer at 0x296c72200>)]
```

Como mostra a saída, `ner` e `parser` não estão mais no pipeline.

1.2 Analisando o pipeline com `analyze_pipes()`

O método `analyze_pipes()` dá uma visão geral dos componentes. Definindo `pretty` como `True`, o `spaCy` imprime uma tabela com os componentes e as anotações que produzem:

Python

```
# Analisa o pipeline e guarda a análise na variável 'pipe_analysis'
pipe_analysis = nlp.analyze_pipes(pretty=True)
```

Saída

```
===== Pipeline Overview =====

#   Component      Assigns      Requires      Scores      Retokenizes
-   -
0   tok2vec        doc.tensor
1   tagger         token.tag      tag_acc
2   attribute_ruler
3   lemmatizer     token.lemma    lemma_acc

[OK] No problems found.
```

O método retorna um **dicionário** Python. Você pode usá-lo para verificar que nenhum problema foi encontrado **antes** de processar grandes volumes. Os relatórios ficam sob a chave `problems`:

Python

```
# Examina o valor guardado sob a chave 'problems'
pipe_analysis['problems']
```

Saída

```
{'tok2vec': [], 'tagger': [], 'attribute_ruler': [], 'lemmatizer': []}
```

Podemos escrever um trecho que **verifica** isso. Percorremos o dicionário com `items()` e usamos `assert` com `len()` e o operador `==` para checar que a lista tem tamanho `0`; caso contrário, o Python levanta um `AssertionError`:

Python

```
# Percorre os pares chave/valor do dicionário. Atribui a chave e o valor
# as variáveis 'component_name' e 'problem_list'.
for component_name, problem_list in pipe_analysis['problems'].items():

    # Usa 'assert' para checar a lista de problemas; levanta Error se necessario.
    assert len(problem_list) == 0, f"There is a problem with {component_name}: {
        problem_list}!"
```

Usamos uma *f-string*: o `f` antes das aspas permite **inserir variáveis** entre chaves `{}`. Se nenhum problema for encontrado, o laço passa silenciosamente.

2. Processando textos com eficiência

Ao trabalhar com grandes volumes, é desejável processá-los da forma mais eficiente possível. Vamos definir um exemplo com uma **lista** de três frases da Wikipédia em inglês:

Python

```
# Reinicializa o modelo de linguagem, pois precisamos da análise de
# dependências nas próximas seções.
nlp = spacy.load('en_core_web_sm')

# Define uma lista de frases de exemplo
sents = ["On October 1, 2009, the Obama administration went ahead with a Bush
         administration program, increasing nuclear weapons production.",
         "The 'Complex Modernization' initiative expanded two existing nuclear
         sites to produce new bomb parts.",
         "The administration built new plutonium pits at the Los Alamos lab in New
         Mexico and expanded enriched uranium processing at the Y-12 facility in Oak
         Ridge, Tennessee."]

# Chama a variável para examinar a saída
sents
```

Os objetos `Language` têm um método específico, `pipe()`, para processar textos em uma **lista**. Ele foi otimizado para processar em **lotes** (*batches*), sendo mais rápido que um laço `for`. O `pipe()` recebe uma lista e retorna um **gerador** (*generator*):

Python

```
# Alimenta a lista de frases ao método pipe()
docs = nlp.pipe(sents)

# Chama a variável para examinar a saída
docs
```

Saída

```
<generator object Language.pipe at 0x296d0b580>
```

Geradores contêm outros objetos e os **produzem** (*yield*) quando chamados. Para recuperar todos, convertamos o gerador em uma **lista**:

Python

```
# Converte o gerador pipe em uma lista
docs = list(docs)

# Chama a variável para examinar a saída
docs
```

Isso nos dá uma lista de objetos `Doc` prontos para processamento posterior.

3. Adicionando atributos personalizados aos objetos do spaCy

O spaCy permite **definir atributos personalizados** para objetos `Doc`, `Span` e `Token` — úteis para guardar informações adicionais sobre os textos. Usamos o método `set_extension()`. Como o atributo é adicionado a **todos** os `Doc`, primeiro importamos o objeto `Doc` genérico:

Python

```
# Importa o objeto Doc do modulo 'tokens' do spaCy
from spacy.tokens import Doc

# Adiciona dois atributos personalizados ao objeto Doc, 'age' e 'location',
# usando o metodo set_extension().
Doc.set_extension("age", default=None)
Doc.set_extension("location", default=None)
```

O argumento `default` define um valor padrão, aqui a palavra-chave `None`.

Nota

Diferentemente de atributos como `sents` ou `heads`, os personalizados ficam sob um atributo que consiste no sublinhado `_` — por exemplo, `Doc._.age`.

Vamos definir um dicionário aninhado `sents_dict` com três chaves (0, 1, 2), cujos valores são dicionários com `age`, `location` e `text`:

Python

```
# Cria um dicionario cujos valores sao outros dicionarios
# com tres chaves: 'age', 'location' e 'text'.
sents_dict = {0: {"age": 23,
                  "location": "Helsinki",
                  "text": "The Senate Square is by far the most important landmark
in Helsinki."},
              1: {"age": 35,
                  "location": "Tallinn",
                  "text": "The Old Town, for sure."},
              2: {"age": 58,
                  "location": "Stockholm",
                  "text": "Södermalm is interesting!"},
              }
```

Percorremos o dicionário, processamos cada texto e definimos os atributos personalizados:

Python

```
# Prepara uma lista vazia para guardar os textos processados
docs = []
```

```

# Percorre os pares de chave e valor do dicionario 'sents_dict'.
# Os pares chave/valor ficam disponiveis pelo metodo items().
# Chamamos essas chaves e valores de 'key' e 'data'; ou seja, usamos
# a variavel 'data' para se referir ao dicionario aninhado.
for key, data in sents_dict.items():

    # Recupera o valor da chave 'text' do dicionario aninhado.
    # Alimenta esse texto ao modelo em 'nlp' e guarda o resultado em 'doc'.
    doc = nlp(data['text'])

    # Recupera os valores de 'age' e 'location' do dicionario aninhado.
    # Atribui esses valores aos atributos personalizados do objeto Doc.
    # Lembre-se: atributos personalizados ficam sob o pseudo-atributo '_'!
    doc._.age = data['age']
    doc._.location = data['location']

    # Acrescenta o Doc atual em 'doc' a lista 'docs'
    docs.append(doc)

```

Isso produz uma lista de **Docs**. Vamos percorrê-la e imprimir cada um com seus atributos:

Python

```

# Percorre cada objeto Doc na lista 'docs'
for doc in docs:

    # Imprime cada Doc e os atributos 'age' e 'location'
    print(doc, doc._.age, doc._.location)

```

Saída

```

The Senate Square is by far the most important landmark in Helsinki. 23 Helsinki
The Old Town, for sure. 35 Tallinn
Södermalm is interesting! 58 Stockholm

```

3.1 Filtrando dados com compreensão de lista

Os atributos personalizados podem ser usados para **filtrar** dados. Uma forma eficiente é a **compreensão de lista** (*list comprehension*) — como um laço **for** declarado “na hora”, entre colchetes `[]`, com três partes: (1) o que guardar na nova lista; (2) o **for ... in** percorrendo os itens; (3) o **if** com a condição:

Python

```

# Usa uma compreensao de lista para filtrar os Docs cujo atributo
# 'age' tenha valor abaixo de 40.
under_forty = [doc for doc in docs if doc._.get('age') < 40]

# Chama a variavel para examinar a saida
under_forty

```

Saída

```
[The Senate Square is by far the most important landmark in Helsinki.,  
The Old Town, for sure.]
```

Isso retorna uma lista com apenas dois `Docs` que atendem ao critério.

4. Gravando textos processados em disco

Com grandes volumes, primeiro garanta que o pipeline funciona com **poucos** textos; depois processe tudo e **salve**. O spaCy oferece o tipo `DocBin` para armazenar objetos `Doc` com suas anotações:

Python

```
# Importa o objeto DocBin do modulo 'tokens' do spaCy  
from spacy.tokens import DocBin  
  
# Inicializa um objeto DocBin e adiciona os Docs de 'docs'  
docbin = DocBin(docs=docs)
```

Atenção

Se você adicionou **atributos personalizados** a `Docs`, `Spans` ou `Tokens`, também é preciso definir `store_user_data=True` — por exemplo, `DocBin(docs=docs, store_user_data=True)`.

Verificamos que os três `Docs` entraram no `DocBin` com o método `__len__()`:

Python

```
# Obtem o numero de Docs no DocBin  
docbin.__len__()
```

Saída

```
3
```

O método `add()` permite adicionar mais `Docs`:

Python

```
# Define uma string, alimenta o modelo em 'nlp' e adiciona o  
# Doc resultante ao objeto DocBin 'docbin'  
docbin.add(nlp("Yet another Doc object."))  
  
# Verifica que o Doc foi adicionado; o tamanho agora deve ser 4  
docbin.__len__()
```

Saída

4

Gravamos o `DocBin` em disco com `to_disk()`, que recebe o argumento `path`:

Python

```
# Grava o objeto DocBin em disco
docbin.to_disk(path='data/docbin.spacy')
```

Para **carregar**, inicialize um `DocBin` vazio e use `from_disk()`:

Python

```
# Inicializa um novo DocBin e usa 'from_disk' para carregar os dados do disco.
# Atribui o resultado a variavel 'docbin_loaded'.
docbin_loaded = DocBin().from_disk(path='data/docbin.spacy')

# Chama a variavel para examinar a saida
docbin_loaded
```

Saída

```
<spacy.tokens._serialize.DocBin at 0x2953a4c40>
```

Para acessar os Docs guardados, use `get_docs()`, passando o **vocabulário** (`nlp.vocab`) necessário para reconstruir os dados:

Python

```
# Usa 'get_docs' para recuperar os Docs do DocBin, passando o
# vocabulario em 'nlp.vocab' para reconstruir os dados.
# Converte o gerador resultante em uma lista para exame.
docs_loaded = list(docbin_loaded.get_docs(nlp.vocab))

# Chama a variavel para examinar a saida
docs_loaded
```

Saída

```
[The Senate Square is by far the most important landmark in Helsinki.,
The Old Town, for sure.,
Södermalm is interesting!,
Yet another Doc object.]
```

Resumindo

O ideal é processar os textos **uma vez**, gravá-los em disco e carregá-los para as análises seguintes.

5. Simplificando a saída: sintagmas nominais e entidades nomeadas

5.1 Mesclando sintagmas nominais

Às vezes é mais vantajoso operar com unidades maiores que tokens individuais, como **sintagmas nominais** (*noun phrases*). O spaCy dá acesso a eles pelo atributo `noun_chunks` de um `Doc`:

Python

```
# Primeiro laço: percorre a lista 'docs'; 'doc' se refere aos itens da lista
for doc in docs:

    # Percorre cada sintagma nominal do objeto Doc
    for noun_chunk in doc.noun_chunks:

        # Imprime o sintagma nominal
        print(noun_chunk)
```

Saída

```
The Senate Square
the most important landmark
Helsinki
The Old Town
Södermalm
```

Para **mesclar** os sintagmas em um único token, o spaCy oferece a função `merge_noun_chunks` adicionada ao pipeline com `add_pipe`:

Python

```
# Adiciona o componente que mescla sintagmas nominais em Tokens unicos
nlp.add_pipe('merge_noun_chunks')
```

Nota

Não é preciso reatribuir o objeto `Language` à mesma variável — o `add_pipe` adiciona o componente automaticamente.

Processando as frases novamente com `pipe()`, ao percorrer os tokens do primeiro `Doc` (`[0]`) vemos os sintagmas **mesclados** e rotulados como **NOUN**:

Python

```
# Aplica o objeto Language 'nlp' a lista de frases em 'sents'
docs = list(nlp.pipe(sents))

# Percorre os Tokens do primeiro Doc da lista
for token in docs[0]:

    # Imprime o Token e sua classe gramatical
    print(token, token.pos_)
```

Saída

```
On ADP
October PROPN
1 NUM
, PUNCT
2009 NUM
, PUNCT
the Obama administration NOUN
went VERB
ahead ADV
with ADP
a Bush administration program NOUN
, PUNCT
increasing VERB
nuclear weapons production NOUN
. PUNCT
```

Rotular como **NOUN** é uma aproximação razoável, já que as palavras-núcleo são substantivos. Mesclar os sintagmas **simplifica** a árvore sintática:

Python

```
displacy.render(docs[0], style='dep')
```

Os sintagmas continuam disponíveis em `noun_chunks` guardados como `Spans` com atributos `start` e `end`.

Python

```
# Percorre os sintagmas nominais do primeiro Doc [0] na lista 'docs'
for noun_chunk in docs[0].noun_chunks:

    # Imprime o sintagma, seu tipo, e os índices onde começa e termina
    print(noun_chunk, type(noun_chunk), noun_chunk.start, noun_chunk.end)
```

Saída

```
October <class 'spacy.tokens.span.Span'> 1 2
the Obama administration <class 'spacy.tokens.span.Span'> 6 7
a Bush administration program <class 'spacy.tokens.span.Span'> 10 11
nuclear weapons production <class 'spacy.tokens.span.Span'> 13 14
```

5.2 Mesclando entidades nomeadas

Entidades nomeadas são mescladas do mesmo modo, com `merge_entities`. Primeiro removemos a função `merge_noun_chunks` com `remove_pipe()`:

Python

```
# Remove a funcao 'merge_noun_chunks' do pipeline em 'nlp'
nlp.remove_pipe('merge_noun_chunks')
```

```
# Processa as frases originais novamente
docs = list(nlp.pipe(sents))
```

Em seguida, adicionamos `merge_entities` e reprocessamos, examinando o terceiro Doc:

```
Python

# Adiciona a funcao 'merge_entities' ao pipeline
nlp.add_pipe('merge_entities')

# Processa os dados novamente
docs = list(nlp.pipe(sents))

# Percorre os Tokens do terceiro Doc da lista
for token in docs[2]:

    # Imprime o Token e sua classe gramatical
    print(token, token.pos_)
```

Saída

```
The DET
administration NOUN
built VERB
new ADJ
plutonium NOUN
pits NOUN
at ADP
the DET
Los Alamos PROPN
lab NOUN
in ADP
New Mexico PROPN
and CCONJ
expanded VERB
enriched ADJ
uranium NOUN
processing NOUN
at ADP
the DET
Y-12 NUM
facility NOUN
in ADP
Oak Ridge PROPN
, PUNCT
Tennessee PROPN
. PUNCT
```

Entidades nomeadas com vários tokens — como os topônimos “*Los Alamos*” e “*New Mexico*” — foram **mescladas** em tokens únicos.

Quiz

Marque a alternativa correta (a certa está marcada com **(correta)**).

1. Por que remover do pipeline componentes que você não usa?

1. Cada componente tem um custo computacional **(correta)**
2. Deixa o código mais bonito
3. É obrigatório

2. Qual argumento de `load()` exclui componentes do pipeline?

1. `exclude` **(correta)**
2. `remove`
3. `skip`

3. Por que o Tokenizer não aparece em `nlp.pipeline`?

1. Ele sempre roda e fica em `nlp.tokenizer` **(correta)**
2. Foi removido do spaCy
3. É opcional e estava desligado

4. Onde ficam os atributos personalizados de um Doc?

1. Sob um atributo que consiste no sublinhado (`Doc._`) **(correta)**
2. Numa coluna extra do texto
3. Dentro de `nlp.meta`

5. Qual método processa uma lista de textos em lote (mais rápido)?

1. `pipe()` **(correta)**
2. `apply()`
3. `map()`

6. Para que serve o DocBin?

1. Guardar e gravar Docs processados em disco **(correta)**
2. Baixar modelos de linguagem
3. Treinar o modelo

Resumo da unidade

Nesta unidade, você aprendeu a

1. examinar o pipeline com `nlp.pipeline` e entender `tok2vec/attribute_ruler`;
2. excluir componentes (`exclude=[...]`) e auditar o pipeline com `analyze_pipes()` e `assert`;
3. processar em lote com `nlp.pipe()`, entendendo geradores e conversão para lista;
4. adicionar atributos personalizados (`set_extension()`, `Doc._`) e filtrar com compreensão de lista;
5. gravar e carregar com `DocBin` (`to_disk()`, `from_disk()`, `get_docs()`);

6. mesclar sintagmas nominais e entidades com `add_pipe()/remove_pipe()`.

Exercícios sugeridos

1. Carregue o modelo excluindo apenas o `lemmatizer` e confirme, com `nlp.pipeline`, que ele não está mais presente.
2. Adicione um atributo personalizado `source` ao `Doc` e preencha-o com o nome do arquivo de origem ao processar textos da Unidade 1.
3. Processe as notícias da Unidade 1 com `nlp.pipe()`, grave em um `DocBin` e recarregue do disco, conferindo o número de `Docs`.
4. Compare a árvore de dependências de uma frase **com** e **sem** `merge_noun_chunks` e descreva a diferença.

Referências. *Applied Language Technology* (MOOC), Universidade de Helsinque — applied-language-technology.mooc.fi
Honnibal, M.; Montani, I. et al. *spaCy: Industrial-strength Natural Language Processing in Python* — spacy.io.

Material produzido para o Programa CiberExt 26-29 / FEELT38103 — Universidade Federal de Uberlândia.